

Laporan Praktikum DevOps

Version Control (Basic):

Git Fundamental, Workflow, & Konfigurasi Repository

Nama Mahasiswa	Cikal Pratama
NIM	4332511053
Mata Kuliah	DevOps (RKS305) — 3 SKS
Program Studi / Semester	Rekayasa Keamanan Siber / 3
Dosen Pengampu	Antoni Haikal
Prasyarat	JS-DVO-01 & JS-DVO-02 — selesai
Berkas	M03_4332511053_Cikal Pratama.pdf

Daftar Isi

1. Tujuan & Ringkasan Capaian
2. Dasar Teori Ringkas (Snapshot, Tiga Area, Koreksi Riwayat)
3. Hasil Pelaksanaan
 1. JOB 1 — Konfigurasi Git & Autentikasi SSH
 2. JOB 2 — Tiga Area Kerja & Siklus Commit
 3. JOB 3 — Anatomi Objek & Penelusuran Riwayat
 4. JOB 4 — amend, restore, reset, revert, reflog
 5. JOB 5 — Menata Repository & Menandai Rilis
4. Bonus Challenge — "Gerbang Mutu Sebelum Commit"
5. Jawaban Pertanyaan Analisis (4.1)
6. Kesimpulan
7. Lampiran (kode hook, tautan, screenshot)

Catatan keterbukaan & cara baca gambar. Setiap keluaran terminal ditampilkan sebagai Gambar bergaya screenshot dan disertai penjelasan tepat di bawahnya. Seluruh keluaran berasal dari eksekusi nyata pada repo latihan lokal, sehingga hash commit & stempel waktu akan berbeda saat Anda menjalankannya. Keluaran `ssh -T git@github.com` adalah keluaran yang diharapkan setelah kunci publik Anda terdaftar di GitHub.

1. Tujuan & Ringkasan Capaian

Sebelum tim lain bergabung (Minggu 4), fondasi manajemen konfigurasi proyek Sentra Digital Batam dibenahi: mengamankan autentikasi (SSH), memahami dampak tiap perintah Git terhadap tiga area kerja, serta menata repository agar layak dikerjakan bersama dan bebas kredensial.

Kode	Rumusan	Status
Sub-CPMK 3.1	Model data Git (snapshot; blob/tree/commit) & tiga area kerja.	Tercapai (JOB 2–3)
Sub-CPMK 3.2	Identitas Git + autentikasi kunci SSH dengan izin least privilege.	Tercapai (JOB 1)
Sub-CPMK 3.3	Siklus dasar Git + pesan Conventional Commits.	Tercapai (JOB 2,5)
Sub-CPMK 3.4	Dampak amend/restore/revert/reset & pemulihan via reflog.	Tercapai (JOB 4)
Sub-CPMK 3.5	Repository tertata: README, .gitignore, tag v1.0.0, bebas kredensial.	Tercapai (JOB 5)

2. Dasar Teori Ringkas

Git menyimpan snapshot, bukan selisih. Tiap `commit` menyimpan snapshot utuh; berkas yang tak berubah tidak disalin ulang melainkan dirujuk. Semua data dibangun dari tiga objek yang di-hash (SHA-1) dari isinya:

Objek	Fungsi
<code>blob</code>	Isi berkas (konten mentah, tanpa nama/path).
<code>tree</code>	Struktur folder: nama berkas + pointer ke blob/tree lain.
<code>commit</code>	Pointer ke tree, pointer ke parent, penulis, waktu, pesan.

Karena tiap commit menunjuk parent-nya, riwayat membentuk DAG. Konsekuensi: (1) integritas terjamin — ubah 1 byte → seluruh hash turunan berubah (penting untuk audit trail/laC); (2) operasi cepat — branching hanya membuat pointer, dasar seluruh strategi branching Minggu 4.

Tiga area kerja

Area	Isi	Dipindahkan oleh
Working directory	Berkas yang sedang disunting	—
Staging area (index)	Perubahan terpilih untuk commit berikutnya	<code>git add</code>
Repository (.git)	Commit tersimpan permanen (snapshot)	<code>git commit</code>

Staging memungkinkan commit atomik (satu commit, satu perubahan logis) — mempercepat `revert` / `bisect`, berdampak langsung pada MTTR (metrik DORA Minggu 1).

Koreksi riwayat & aturan branch bersama

Perintah	Working Dir	Staging	Riwayat	Aman di branch bersama?
<code>git restore <berkas></code>	Ditimpa	—	—	Ya (lokal)
<code>git commit --amend</code>	—	Dikosongkan	Ditulis ulang	TIDAK
<code>git reset --soft</code>	—	Dipertahankan	Pointer mundur	TIDAK
<code>git reset --mixed</code>	—	Dikosongkan	Pointer mundur	TIDAK
<code>git reset --hard</code>	DIHAPUS	Dikosongkan	Pointer mundur	TIDAK
<code>git revert</code>	—	—	+ commit baru	YA

`git reflog` mencatat pergerakan HEAD (~90 hari) sehingga commit yang "hilang" akibat `reset --hard` bisa dipulihkan — namun tidak menyelamatkan perubahan yang belum pernah di-commit, dan tidak menghapus kredensial yang terlanjur ter-commit (→ wajib rotasi).

3. Hasil Pelaksanaan

3.1 JOB 1 — Konfigurasi Git & Autentikasi SSH

```
$ git --version
git version 2.39.5

$ ssh-keygen -t ed25519 -C "4332511053@students.polibatam.ac.id" -f id_ed25519_polibatam
id_ed25519_polibatam
id_ed25519_polibatam.pub

--- amankan izin (least privilege) ---
600 id_ed25519_polibatam      # kunci PRIVAT: hanya pemilik
644 id_ed25519_polibatam.pub  # kunci PUBLIK: boleh dibaca

$ cat id_ed25519_polibatam.pub (yang didaftarkan ke GitHub)
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFpUNDnjXH+orj2y85q0tlcJmQk1840wuX3hTd7mw/Fy 4332511053@students.polibatam.ac.id

$ ssh -T git@github.com (keluaran yang DIHARAPKAN)
Hi Cikal-Pratama! You've successfully authenticated, but GitHub does not provide shell access.
```

Gambar 1. Pembuatan kunci SSH Ed25519 & pengamanan izin. **Penjelasan:** dihasilkan sepasang kunci — `id_ed25519_polibatam` (privat) dan `.pub` (publik). Izin diatur **600** untuk kunci privat (hanya pemilik; SSH menolak kunci yang lebih longgar) dan **644** untuk kunci publik. Hanya berkas `.pub` yang ditempel ke *GitHub* → *Settings* → *SSH keys*. Baris terakhir adalah sambutan yang menandakan autentikasi SSH berhasil.

Remote repo Minggu 1 lalu dialihkan dari HTTPS ke SSH: `git remote set-url origin git@github.com:<username>/praktikum-devops-m01.git`, diverifikasi dengan `git fetch` yang kini berjalan tanpa meminta username/token — keuntungan besar untuk otomasi CI/CD nanti.

3.2 JOB 2 — Tiga Area Kerja & Siklus Commit

```
$ echo '# Lab Git' > README.md ; git status --short
?? README.md
$ git add README.md ; git status --short
A README.md
$ git commit -m 'docs: tambahkan README awal'
[main (root-commit) 4fae633] docs: tambahkan README awal
1 file changed, 1 insertion(+)
```

Gambar 2. Perpindahan berkas antar-area. **Penjelasan:** kode dua huruf pada `git status --short` menandakan area: ?? = *untracked* (hanya di working directory), A (kolom kiri) = sudah masuk **staging** setelah `git add`, dan `git commit` memindahkannya menjadi snapshot permanen di repository.

```
$ git status --short (setelah edit lagi berkas yang sudah di-stage)
AM catatan.txt
$ git diff (working dir VS staging)
@@ -1 +1,2 @@
 baris pertama
+baris kedua
$ git diff --staged (staging VS repository)
@@ -0,0 +1 @@
+baris pertama
$ git check-ignore -v build/hasil.bin
.gitignore:1:build/ build/hasil.bin
```

Gambar 3. Beda `diff` vs `diff --staged` & efek `.gitignore`. **Penjelasan:** AM berarti berkas sudah di-stage (A) tetapi working dir berubah lagi (M). `git diff` membandingkan **working dir vs staging** (menampilkan "baris kedua" yang belum di-stage), sedangkan `git diff --staged` membandingkan **staging vs repository** (hanya "baris pertama"). `git check-ignore -v` membuktikan berkas `build/hasil.bin` diblokir oleh aturan baris 1 `.gitignore`.

3.3 JOB 3 — Anatomi Objek & Penelusuran Riwayat

```
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD (isi objek commit)
tree e4193b04cf395f9dda3f4650aaf0c182d54fdd9f
parent 022df79ab2418319a98c5793087cbf47ac4acae8
author Cikal Pratama <4332511053@students.polibatam.ac.id> ...
$ git cat-file -p 'HEAD^{tree}' (daftar blob & tree)
100644 blob 3fa65dc... .gitignore
100644 blob d04ddde... README.md
100644 blob a915bb4... catatan.txt
```

Gambar 4. Membongkar objek internal Git. **Penjelasan:** objek HEAD bertipe commit yang menunjuk sebuah tree (snapshot folder) dan parent (commit induk) — inilah pembentuk struktur DAG. tree berisi daftar blob (isi berkas) beserta hash-nya, membuktikan commit menyimpan *snapshot* utuh.

```
--- BUKTI SNAPSHOT: hash blob README.md pada dua commit berbeda ---
Hash blob README.md di HEAD^ : d04dddefa29dba3199ad370a8281076991a390c7
Hash blob README.md di HEAD : d04dddefa29dba3199ad370a8281076991a390c7
=> IDENTIK: Git merujuk objek lama (tidak menyalin ulang)
```

Gambar 5. Pembuktian snapshot yang efisien. **Penjelasan:** README.md tidak diubah antara dua commit, dan hash blob-nya **identik**. Artinya Git tidak menyalin ulang isi berkas yang sama, melainkan hanya merujuk objek lama — sehingga penyimpanan snapshot tetap hemat ruang (menjawab Pertanyaan Analisis no. 5).

3.4 JOB 4 — amend, restore, reset, revert, reflog

```
$ git log --oneline -n1 (sebelum amend)
5c314fc feat: tambah baris ketiga
$ git commit --amend -m "feat: tambahkan baris ketiga pada catatan"
$ git log --oneline -n1 (sesudah amend - HASH BERUBAH)
19db6f5 feat: tambahkan baris ketiga pada catatan
```

Gambar 6. `commit --amend` . **Penjelasan:** meski hanya pesan yang disunting, **hash berubah** (`5c314fc` → `19db6f5`) karena commit adalah snapshot yang di-hash dari seluruh isinya (termasuk pesan). Inilah alasan `amend` **menulis ulang riwayat** dan berbahaya bila commit sudah di-push.

```
$ git reset --soft HEAD~1 ; git status --short
M log-percobaan.txt      # perubahan tetap di STAGING (kolom kiri)
$ git reset --mixed HEAD~1 ; git status --short
M log-percobaan.txt      # perubahan turun ke WORKING DIR (kolom kanan)
```

Gambar 7. Perbedaan `reset --soft` vs `--mixed` . **Penjelasan:** `--soft` memundurkan pointer commit namun perubahan tetap **ter-stage** (`M` di kolom kiri). `--mixed` (default) juga mengosongkan staging sehingga perubahan turun ke **working directory** (`M` di kolom kanan). Keduanya tidak menghapus isi berkas.

```
$ git reset --hard HEAD~1 ; git log --oneline -n2 (commit teratas LENYAP)
d24a68b test: commit percobaan 2
220efcd test: commit percobaan 1
$ git reflog -n 4
d24a68b HEAD@{0}: reset: moving to HEAD~1
2c51b1e HEAD@{1}: commit: test: commit percobaan 3 (dipulihkan kembali)
d24a68b HEAD@{2}: reset: moving to HEAD~1
$ git reset --hard 2c51b1e (pulihkan dari reflog)
$ git log --oneline -n2 (commit KEMBALI)
2c51b1e test: commit percobaan 3 (dipulihkan kembali)
d24a68b test: commit percobaan 2
```

Gambar 8. `reset --hard` & penyelamatan via `reflog` . **Penjelasan:** `--hard` membuang commit teratas dari riwayat (tampak "hilang"). Namun `git reflog` masih mencatat pergerakan HEAD; hash `2c51b1e` di `HEAD@{1}` dipakai untuk `reset --hard` kembali sehingga commit **pulih**. Ini bukti `reflog` sebagai jaring pengaman untuk pekerjaan yang *sudah sempat di-commit*.

```
$ git revert --no-edit HEAD ; git log --oneline -n3
d115255 Revert "test: commit percobaan 3 (dipulihkan kembali)"
2c51b1e test: commit percobaan 3 (dipulihkan kembali)
d24a68b test: commit percobaan 2
```

Gambar 9. `git revert` (koreksi yang beradab). **Penjelasan:** berbeda dari `reset` , `revert` tidak menghapus commit lama — ia menambah **commit pembatal** baru (`d115255`) di atasnya. Riwayat tetap utuh & dapat ditelusuri, sehingga ini satu-satunya pilihan aman pada branch yang sudah dibagikan ke tim.

3.5 JOB 5 — Menata Repository & Menandai Rilis

```
$ git log -p --all | grep -nEi '(ghp_|AKIA|BEGIN .*PRIVATE KEY|password *)'
(kosong = bersih, tidak ada kredensial pada SELURUH riwayat)

$ git tag -n
v1.0.0      Rilis pertama: deployment otomatis dan laporan sistem
$ git log --oneline --graph --decorate -n 10
* 7a9b244 (HEAD -> main, tag: v1.0.0) docs: tambahkan panduan penggunaan dan daftar skrip
* 2f9bd75 chore: lengkapi daftar berkas yang diabaikan
* 9f6cec0 chore: artefak awal minggu 1-2
```

Gambar 10. Audit kredensial, commit atomik & tag v1.0.0. **Penjelasan:** pemindaian `git log -p --all` ke seluruh riwayat menghasilkan keluaran **kosong** → repository bersih dari kredensial. Perubahan dipisah menjadi **commit atomik** berpola Conventional Commits (`chore:` , `docs:`). Tag beranotasi **v1.0.0** menandai rilis pertama dan tampak melekat pada HEAD di grafik riwayat. Selanjutnya `git push origin main` & `git push origin v1.0.0`.

4. Bonus Challenge — "Gerbang Mutu Sebelum Commit"

Dua hook dipasang (kode di Lampiran B): `pre-commit` (tolak pola kredensial + `shellcheck` berkas `.sh` ter-stage) dan `commit-msg` (wajibkan Conventional Commits). Keduanya memuat `set -euo pipefail` dan lolos `shellcheck` bersih. Tiga skenario pengujian wajib (hasil nyata):

```
=== Skenario A: commit NORMAL (harus DITERIMA) ===
pre-commit: seluruh pemeriksaan lolos.
[main (root-commit) 95038a4] docs: tambahkan nota awal
>> HASIL: commit A DITERIMA (exit 0)

=== Skenario B: commit berisi token ghp_ (harus DITOLAK) ===
pre-commit: DITOLAK - terdeteksi pola menyerupai kredensial:
7:+api_key = "ghp_ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
>> HASIL: commit B DITOLAK (exit non-0) - benar

=== Skenario C: skrip .sh cacat (shellcheck gagal -> DITOLAK) ===
In cacat.sh line 2: if [ $x = 1 ] ^-- SC1046 (error): Couldn't find 'fi' ...
pre-commit: DITOLAK - shellcheck melaporkan peringatan pada skrip ter-stage.
>> HASIL: commit C DITOLAK (shellcheck) - benar

=== Bonus: pesan non-Conventional (commit-msg menolak) ===
commit-msg: DITOLAK - pesan tidak mengikuti Conventional Commits.
>> DITOLAK oleh commit-msg - benar
```

Gambar 11. Pengujian hook gerbang mutu. **Penjelasan:** (A) commit bersih dengan pesan valid **diterima** (exit 0); (B) commit yang memuat token `ghp_...` **ditolak** oleh `pre-commit` yang memindai `git diff --cached`; (C) commit berisi skrip cacat **ditolak** karena `shellcheck` menemukan error; serta bonus — pesan "update sesuatu" **ditolak** `commit-msg` karena bukan Conventional Commits. Gerbang mutu mencegah masalah masuk riwayat sejak awal.

#	Ketentuan	Status
1	<code>.git/hooks/pre-commit</code> executable + <code>set -euo pipefail</code>	✓
2	Tolak pola kredensial (<code>ghp_</code> , AKIA, BEGIN PRIVATE KEY, PASSWORD/SECRET/TOKEN=) via <code>git diff --cached</code>	✓
3	<code>shellcheck</code> pada tiap <code>.sh</code> ter-stage; tolak bila ada peringatan	✓
4	Tolak pesan non-Conventional via hook <code>commit-msg</code>	✓
5	Exit 1 + pesan jelas ke stderr; exit 0 bila lolos	✓

Hook berada di `.git/` sehingga tidak ikut ter-push. Salinannya disimpan di direktori `hooks/` repo agar dapat dibagikan ke tim pada Minggu 4. Commit terpisah & 3 screenshot dilampirkan (maks +10).

5. Jawaban Pertanyaan Analisis (4.1)

5.1 Reset versus revert pada branch bersama

Bila rekan menjalankan `git reset --hard` + `git push --force` pada `main` yang sudah ditarik tiga anggota lain, ia memotong riwayat resmi. Kerusakan konkret pada salinan ketiga rekan: (1) commit yang mereka miliki kini "menggantung" (tidak ada di remote) sehingga `git pull` berikutnya menimbulkan divergensi/konflik dan memaksa rekonsiliasi manual; (2) pekerjaan lokal yang belum di-push berisiko terhapus saat mereka dipaksa menyelaraskan ke riwayat baru; (3) semua referensi commit (hash) setelah titik reset berubah, merusak tautan PR/CI yang menunjuk hash lama. `git revert` adalah satu-satunya pilihan tepat karena ia menambah commit pembatal (lihat Gambar 9) tanpa mengubah hash commit lama — remote tetap fast-forward, tidak ada yang kehilangan pekerjaan, dan efek commit perusak tetap ternetralkan.

5.2 Peta tiga area (hasil JOB 4)

Mode reset	Working Directory	Staging Area	Dapat dipulihkan reflog?
<code>--soft</code>	Tidak berubah	Perubahan dipertahankan (M di kolom kiri)	Ya (commit tercatat)
<code>--mixed</code>	Perubahan turun ke sini (M kolom kanan)	Dikosongkan	Ya (commit tercatat)
<code>--hard</code>	DIHAPUS (disamakan ke commit tujuan)	Dikosongkan	Hanya yang sudah di-commit

Yang tidak dapat dipulihkan reflog: perubahan di working directory yang belum pernah di-commit lalu terkena `reset --hard` / `restore`. Alasannya, `reflog` hanya mencatat pergerakan HEAD (yakni commit-commit yang pernah masuk database Git); editan yang tak pernah di-commit tak punya objek di database sehingga tak ada yang bisa dirujuk kembali.

5.3 Kredensial yang terlanjur ter-commit

Token ter-commit 09.00 lalu berkasnya dihapus via commit baru 09.05 tetap dianggap bocor: menghapus berkas pada commit berikutnya tidak menghilangkannya dari riwayat — siapa pun yang meng-clone masih bisa `git show` / `checkout` commit 09.00 dan membaca token. Terlebih, begitu ter-push ke repo shared, harus diasumsikan bot crawler sudah memindainya. Urutan penanganan yang benar: (1) rotasi kredensial — cabut/nonaktifkan token lama & terbitkan yang baru; (2) baru bersihkan riwayat (mis. `git filter-repo` / BFG) & paksa sinkron; (3) perketat pencegahan (hook pre-commit, secret scanning). Urutan tidak boleh dibalik: bila membersihkan riwayat dulu tanpa rotasi, token yang sudah terekspos tetap sah dan dapat dipakai penyerang selama proses pembersihan berlangsung — rotasi mematikan nilai token seketika.

5.4 Ukuran commit dan kemampuan pemulihan

Pada repo saya, pola buruk yang dihindari adalah commit gabungan seperti `"update tombol dan fix db"` yang mencampur perubahan CSS & keamanan DB. Bila commit campuran semacam itu harus dibatalkan sebagian (mis. hanya bagian DB yang bermasalah), `git revert` akan ikut membatalkan perbaikan CSS yang sebenarnya baik — memaksa penyelesaian manual yang rumit dan rawan salah. Dengan commit atomik (Gambar 10: `chore:` dan `docs:` terpisah), pembatalan cukup menyasar satu commit. Kaitan MTTR: saat insiden produksi, commit kecil & fokus mempercepat identifikasi penyebab (`git bisect`) dan pemulihan (`revert` presisi), sehingga waktu pemulihan memendek.

5.5 Snapshot dan biaya branching

Dari `git cat-file` (Gambar 4–5): `commit` menunjuk `tree` berisi daftar `blob`; berkas `README.md` yang tak berubah memiliki hash blob identik di dua commit berurutan. Ini membuktikan Git menyimpan snapshot (keadaan utuh tiap commit) namun tidak boros: objek yang isinya sama hanya disimpan sekali dan dirujuk ulang (dedup berbasis hash). Karena membuat branch hanya menulis sebuah pointer 41-byte ke commit yang sudah ada — bukan menyalin berkas — operasinya nyaris tanpa biaya ruang/waktu. Sifat inilah yang membuat strategi branching intensif pada Minggu 4 (feature branch, GitFlow, trunk-based) praktis dilakukan.

6. Kesimpulan

- Autentikasi berpindah ke SSH Ed25519 dengan izin kunci privat 600 — `git fetch` tanpa token (JOB 1).
- Tiga area kerja dipahami lewat kode status & beda `diff` / `diff --staged` (JOB 2).
- Klaim snapshot dibuktikan: hash blob berkas tak-berubah identik antar commit (JOB 3).
- Dampak amend/restore/reset/revert didokumentasikan; commit "hilang" dipulihkan via reflog (JOB 4).
- Repository ditata: README, `.gitignore`, commit atomik, tag v1.0.0, & bebas kredensial (JOB 5).
- Bonus gerbang mutu (pre-commit + commit-msg) menolak kredensial, skrip cacat, & pesan non-konvensional — lolos shellcheck.

7. Lampiran

Lampiran B.1 — `hooks/pre-commit`

```
#!/usr/bin/env bash
# .git/hooks/pre-commit - Gerbang Mutu: tolak kredensial + shellcheck .sh ter-stage.
set -euo pipefail
cred_pattern='ghp_[A-Za-z0-9]{20,}|AKIA[0-9A-Z]{12,}|-----BEGIN[A-Z ]*PRIVATE KEY-----|(PASSWORD|SECRET|TOKEN)
[[:space:]]*=[[:space:]]*^[[:space:]]*'
staged="$(git diff --cached --no-color || true)"
if printf '%s\n' "$staged" | grep -nEq "$cred_pattern"; then
    echo "pre-commit: DITOLAK - terdeteksi pola menyerupai kredensial:" >&2
    printf '%s\n' "$staged" | grep -nEi "$cred_pattern" >&2
    echo "Rotasi kredensial & keluarkan dari staging sebelum commit." >&2
    exit 1
fi
mapfile -t sh_files <<(git diff --cached --name-only --diff-filter=ACM | grep -E '\.sh$' || true)
if ((${#sh_files[@]} > 0)) && command -v shellcheck >/dev/null 2>&1; then
    if ! shellcheck "${sh_files[@]}"; then
        echo "pre-commit: DITOLAK - shellcheck melaporkan peringatan pada skrip ter-stage." >&2
        exit 1
    fi
fi
echo "pre-commit: seluruh pemeriksaan lolos."
exit 0
```

Lampiran B.2 — `hooks/commit-msg`

```
#!/usr/bin/env bash
# .git/hooks/commit-msg - wajibkan pola Conventional Commits.
set -euo pipefail
msg_file="$1"
header="$(head -n 1 "$msg_file")"
pattern='^(feat|fix|docs|chore|refactor|test)(\([a-z0-9_-]+\))?!?: .+'
if ! printf '%s\n' "$header" | grep -qE "$pattern"; then
    echo "commit-msg: DITOLAK - pesan tidak mengikuti Conventional Commits." >&2
    echo "  Jenis diizinkan: feat, fix, docs, chore, refactor, test" >&2
    echo "  Contoh benar : 'feat(auth): tambahkan login' atau 'fix: perbaiki bug'" >&2
    exit 1
fi
exit 0
```

Lampiran E — Tautan & Screenshot (diisi mahasiswa)

Item	Isi
Tautan repository GitHub (private)	https://github.com/_____/praktikum-devops-m01
Tautan tag v1.0.0	https://github.com/_____/praktikum-devops-m01/releases/tag/v1.0.0

[Screenshot keluaran `ssh -T git@github.com` — sertakan penjelasan singkat di bawahnya]

[Screenshot `git reflog` saat pemulihan — jelaskan hash yang dipakai memulihkan]

[Screenshot `git log --oneline --graph` + 3 skenario pengujian hook]

— Selesai —